



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

IC-6600 Principios de Sistemas Operativos

Tarea 03: Administración de Memoria

Estudiantes:

Anthony Barrantes Jimenez – 2023152240

Samir Fernando Cabrera Tabash – 2022161229

Deyton Hernandez Cordoba – 2023036696

Profesor:

Kenneth Roberto Obando Rodríguez

16 de noviembre de 2025

II Semestre 2025

Índice

1. Introducción	3
1.1. Objetivos del Proyecto	3
2. Arquitectura del Sistema	3
2.1. Módulos del Sistema	4
2.1.1. main.c - Punto de Entrada	4
2.1.2. memsim.c - Motor del Simulador	4
2.1.3. block.c - Gestión de Bloques	4
2.1.4. varmap.c - Mapeo de Variables	5
2.1.5. util.c - Utilidades Generales	5
3. Estructuras de Datos	5
3.1. BlockHeader - Descriptor de Bloque	5
3.2. VarMap - Mapeo Variable-Bloque	6
3.3. MemSim - Estado del Simulador	6
4. Algoritmos de Asignación	6
4.1. First-Fit (Primer Ajuste)	6
4.2. Best-Fit (Mejor Ajuste)	7
4.3. Worst-Fit (Peor Ajuste)	7
5. Operaciones Implementadas	8
5.1. ALLOC - Asignación de Memoria	8
5.2. REALLOC - Reasignación de Memoria	8
5.3. FREE - Liberación de Memoria	9
5.4. PRINT - Visualización del Estado	9
6. Coalescencia de Bloques	9
7. Expansión Dinámica del Pool	10
8. Formato del Archivo de Entrada	11
9. Compilación y Ejecución	12
9.1. Sistema de Compilación	12
9.2. Uso del Programa	12
10. Resultados y Pruebas	13
10.1. Diseño Experimental	13

10.2. Prueba 1: Estrategia First-Fit	13
10.3. Prueba 2: Estrategia Best-Fit	15
10.4. Prueba 3: Estrategia Worst-Fit	17
11. Conclusiones	19

1. Introducción

Este documento presenta la implementación de un simulador de gestión dinámica de memoria en lenguaje C, desarrollado como parte de la Tarea 3 del curso IC-6600 Principios de Sistemas Operativos. El simulador replica el comportamiento de las funciones estándar de memoria dinámica (`malloc`, `calloc`, `realloc` y `free`) operando sobre un único bloque de memoria preasignado.

El objetivo principal del proyecto es demostrar el funcionamiento interno de la administración de memoria, incluyendo la implementación de tres estrategias clásicas de asignación (First-Fit, Best-Fit y Worst-Fit), así como la simulación de problemas comunes como la fragmentación externa y las fugas de memoria.

1.1. Objetivos del Proyecto

- Implementar un gestor de memoria que opere sobre un pool de bytes preasignado.
- Desarrollar tres algoritmos de asignación de memoria: First-Fit, Best-Fit y Worst-Fit.
- Simular operaciones de asignación (`ALLOC`), reasignación (`REALLOC`) y liberación (`FREE`) de memoria.
- Demostrar problemas de fragmentación de memoria y fugas de memoria.
- Implementar expansión dinámica del pool de memoria utilizando `realloc`.

2. Arquitectura del Sistema

El simulador está organizado en módulos independientes que facilitan el mantenimiento y la escalabilidad del código. La estructura de directorios del proyecto es la siguiente:

```
Code/  
src/ # Archivos fuente (.c)  
    main.c  
    memsim.c  
    block.c  
    varmap.c  
    util.c  
include/ # Archivos de cabecera (.h)  
    memsim.h
```

```
block.h
varmap.h
util.h
build/ # Archivos objeto (.o)
Makefile # Sistema de compilación
entrada.txt # Archivo de prueba
```

2.1. Módulos del Sistema

2.1.1. main.c - Punto de Entrada

El módulo principal se encarga de:

- Validar los argumentos de línea de comandos (estrategia, tamaño del pool, archivo de entrada).
- Inicializar la estructura del simulador de memoria.
- Invocar el procesamiento del script de operaciones.
- Liberar todos los recursos al finalizar.

2.1.2. memsim.c - Motor del Simulador

Contiene la lógica principal del simulador:

- **Inicialización:** Asigna el pool de memoria inicial usando `calloc`.
- **Expansión:** Implementa crecimiento dinámico del pool con `realloc`.
- **Estrategias de asignación:** Implementa First-Fit, Best-Fit y Worst-Fit.
- **Operaciones de memoria:** `do_alloc`, `do_realloc`, `do_free`.
- **Visualización:** Función `print_state` para mostrar el estado de la memoria.
- **Parser:** Procesa el archivo de entrada línea por línea.

2.1.3. block.c - Gestión de Bloques

Maneja la lista doblemente enlazada de bloques de memoria:

- Creación de nuevos bloques (`new_block`).

- Inserción y eliminación de bloques en la lista.
- Coalescencia de bloques libres adyacentes (`try_coalesce`).

2.1.4. varmap.c - Mapeo de Variables

Implementa un diccionario simple que asocia nombres de variables con sus bloques de memoria:

- Búsqueda de variables (`find_var`).
- Registro de nuevas variables (`set_var`).
- Eliminación de variables (`unset_var`).

2.1.5. util.c - Utilidades Generales

Funciones auxiliares del sistema:

- Manejo de errores (`die`).
- Asignación de memoria con verificación (`xmalloc`, `xstrdup`).
- Parseo de estrategias desde cadenas de texto.

3. Estructuras de Datos

3.1. BlockHeader - Descriptor de Bloque

Cada bloque de memoria (libre u ocupado) está representado por la siguiente estructura:

```
typedef struct BlockHeader {
    size_t offset; // Posicin en el pool
    size_t size; // Tamao del bloque
    int free; // 1 = libre, 0 = ocupado
    struct BlockHeader *prev; // Bloque anterior
    struct BlockHeader *next; // Bloque siguiente
} BlockHeader;
```

Esta lista doblemente enlazada permite:

- Recorridos eficientes en ambas direcciones.

- Coalescencia rápida de bloques adyacentes.
- Inserción y eliminación en tiempo constante.

3.2. VarMap - Mapeo Variable-Bloque

Asocia nombres de variables con sus bloques correspondientes:

```
typedef struct VarMap {  
    char *name; // Nombre de la variable  
    BlockHeader *block; // Bloque asignado  
    struct VarMap *next; // Siguiente variable  
} VarMap;
```

3.3. MemSim - Estado del Simulador

Estructura principal que mantiene el estado completo del sistema:

```
typedef struct {  
    uint8_t *pool; // Pool de memoria  
    size_t pool_size; // Tamao actual del pool  
    FitStrategy strat; // Estrategia de asignacin  
    VarMap *vars; // Lista de variables  
    BlockHeader *blocks; // Lista de bloques  
} MemSim;
```

4. Algoritmos de Asignación

4.1. First-Fit (Primer Ajuste)

Busca el primer bloque libre que sea suficientemente grande para satisfacer la solicitud.

Ventajas:

- Rápido: termina en cuanto encuentra un bloque adecuado.
- Simple de implementar.

Desventajas:

- Tiende a fragmentar la memoria al principio del pool.

- Puede dejar bloques pequeños inutilizables.

4.2. Best-Fit (Mejor Ajuste)

Busca el bloque libre más pequeño que pueda satisfacer la solicitud, minimizando el espacio desperdiciado.

Ventajas:

- Reduce el desperdicio de memoria.
- Aprovecha mejor los bloques existentes.

Desventajas:

- Más lento: debe recorrer toda la lista de bloques.
- Puede generar muchos fragmentos pequeños.

4.3. Worst-Fit (Peor Ajuste)

Selecciona el bloque libre más grande disponible, con la intención de dejar fragmentos más grandes y útiles.

Ventajas:

- Los fragmentos residuales son más grandes y potencialmente reutilizables.

Desventajas:

- Consume los bloques grandes rápidamente.
- Puede no ser óptimo para cargas de trabajo con tamaños variados.

```
static BlockHeader* pick_block(MemSim *ms, size_t need) {
    BlockHeader *best = NULL, *worst = NULL;
    for (BlockHeader *b = ms->blocks; b; b = b->next) {
        if (!b->free || b->size < need) continue;
        if (ms->strat == FIRST_FIT) return b;
        if (ms->strat == BEST_FIT) {
            if (!best || b->size < best->size) best = b;
        } else if (ms->strat == WORST_FIT) {
            if (!worst || b->size > worst->size) worst = b;
        }
    }
}
```



```
return (ms->strat == BEST_FIT) ? best : worst;
}
```

5. Operaciones Implementadas

5.1. ALLOC - Asignación de Memoria

Asigna un bloque de memoria de tamaño especificado y lo asocia a una variable.

Proceso:

1. Busca un bloque libre adecuado según la estrategia configurada.
2. Si no hay espacio, expande el pool dinámicamente.
3. Divide el bloque libre si es necesario (split).
4. Rellena la memoria con el nombre de la variable.
5. Registra la variable en el mapa.

Ejemplo de uso:

```
ALLOC A 100
```

5.2. REALLOC - Reasignación de Memoria

Cambia el tamaño de un bloque previamente asignado.

Casos manejados:

- **Mismo tamaño:** No hace nada, solo rellena con el nombre.
- **Reducción:** Reduce el bloque y crea un bloque libre con el espacio sobrante.
- **Expansión contigua:** Si el siguiente bloque es libre y tiene espacio suficiente, lo absorbe.
- **Reubicación:** Si no hay espacio contiguo, expande el pool, busca nuevo espacio, copia los datos y libera el bloque original.

Corrección crítica implementada: En versiones anteriores, el código entraba en bucle infinito al intentar llamar recursivamente a `do_alloc` con el mismo nombre de variable. La

solución implementada realiza la búsqueda de hueco y asignación manualmente, copiando los datos con `memcpy` antes de liberar el bloque antiguo.

Ejemplo de uso:

```
REALLOC B 260
```

5.3. FREE - Liberación de Memoria

Marca un bloque como libre e intenta fusionarlo con bloques libres adyacentes (coalescencia).

Proceso:

1. Busca la variable en el mapa.
2. Marca el bloque como libre.
3. Intenta fusionarlo con bloques libres anteriores y posteriores.
4. Elimina la variable del mapa.

Ejemplo de uso:

```
FREE A
```

5.4. PRINT - Visualización del Estado

Muestra el estado actual de la memoria, incluyendo:

- Tamaño total del pool.
- Lista de todos los bloques (offset, tamaño, estado).
- Variables activas y sus asignaciones.

6. Coalescencia de Bloques

La coalescencia es fundamental para reducir la fragmentación externa. Cuando un bloque se libera, el sistema intenta fusionarlo con bloques libres adyacentes.

```
void try_coalesce(MemSim *ms, BlockHeader *b) {  
    // Fusin con bloque anterior  
    if (b->prev && b->prev->free &&
```

```
        b->prev->offset + b->prev->size == b->offset) {
        b->prev->size += b->size;
        remove_block(ms, b);
        b = b->prev;
    }
    // Fusin con bloque siguiente
    if (b->next && b->next->free &&
        b->offset + b->size == b->next->offset) {
        b->size += b->next->size;
        remove_block(ms, b->next);
    }
}
```

Beneficios:

- Reduce la fragmentación externa.
- Permite reutilizar bloques grandes.
- Mejora la eficiencia del sistema a largo plazo.

7. Expansión Dinámica del Pool

Cuando no hay espacio suficiente para una asignación, el simulador expande automáticamente el pool utilizando `realloc`.

```
void expand_pool(MemSim *ms, size_t extra_bytes) {
    uint8_t *newpool = realloc(ms->pool,
                               ms->pool_size + extra_bytes);
    if (!newpool) die("realloc fall al expandir pool");

    size_t old_size = ms->pool_size;
    ms->pool = newpool;
    ms->pool_size += extra_bytes;

    BlockHeader *last = ms->blocks;
    while (last && last->next) last = last->next;

    if (last && last->free) {
        last->size += extra_bytes;
    } else {
        BlockHeader *b = new_block(old_size, extra_bytes, 1);
```

```
        insert_after(last, b);  
    }  
}
```

Estrategia de crecimiento: El pool crece en múltiplos del tamaño solicitado (factor de 2x) para minimizar el número de expansiones.

8. Formato del Archivo de Entrada

El simulador procesa archivos de texto con las siguientes reglas:

- Una operación por línea.
- Líneas que comienzan con `#` son comentarios.
- Líneas en blanco se ignoran.
- Formato de operaciones: `OPERACION variable tamaño`

Ejemplo de archivo de entrada:

```
# Simulacin de fragmentacin  
ALLOC A 100  
ALLOC B 200  
FREE A  
ALLOC C 50  
PRINT  
  
# Simulacin de fuga de memoria  
ALLOC D 300  
ALLOC E 400  
FREE E  
PRINT  
  
# Pruebas de REALLOC  
REALLOC B 260  
PRINT  
REALLOC C 30  
PRINT
```

9. Compilación y Ejecución

9.1. Sistema de Compilación

El proyecto utiliza un Makefile que automatiza el proceso de compilación:

```
# Compilar el proyecto
make

# Limpiar archivos generados
make clean

# Compilar y ejecutar con First-Fit
make run

# Ejecutar con diferentes estrategias
make run-first
make run-best
make run-worst

# Comparar todas las estrategias
make test-all
```

9.2. Uso del Programa

Sintaxis:

```
./memsim <estrategia> <tamao_pool> <archivo_entrada>
```

Parámetros:

- `estrategia`: first, best o worst
- `tamaño_pool`: Tamaño inicial del pool en bytes
- `archivo_entrada`: Ruta al archivo con las operaciones

Ejemplos:

```
./memsim first 1024 entrada.txt
./memsim best 2048 entrada.txt
./memsim worst 512 entrada.txt
```

10. Resultados y Pruebas

Esta sección presenta los resultados experimentales del simulador de memoria ejecutado con las tres estrategias de asignación implementadas. Se utilizó el mismo archivo de entrada para todas las pruebas, variando únicamente la estrategia y el tamaño inicial del pool, con el objetivo de analizar el comportamiento diferenciado de cada algoritmo.

10.1. Diseño Experimental

Para evaluar de manera exhaustiva el comportamiento del simulador, se diseñó un conjunto de pruebas que incluye:

- Asignaciones consecutivas de diferentes tamaños
- Liberaciones intercaladas para generar fragmentación
- Operaciones de reasignación que requieren expansión de bloques
- Escenarios que provocan expansión automática del pool

El archivo de entrada ejecuta la siguiente secuencia de operaciones:

1. Asignación inicial de tres variables (A, B, C) con tamaños de 100, 200 y 50 bytes respectivamente
2. Liberación de la variable A para crear un hueco en la memoria
3. Reasignación en el hueco liberado (variable C ocupa parcialmente el espacio de A)
4. Estado intermedio mostrando fragmentación
5. Asignación de variable D (300 bytes) y E (400 bytes)
6. Liberación de E
7. Estado mostrando fuga de memoria (D permanece asignada)
8. Reasignación de B aumentando su tamaño (de 200 a 260 bytes)
9. Reasignación de C reduciendo su tamaño (de 50 a 30 bytes)

10.2. Prueba 1: Estrategia First-Fit

La primera prueba ejecuta el simulador con la estrategia First-Fit sobre un pool inicial de 1024 bytes.

```

samircabrera@Samirs-Mac-mini Code % ./memsim first 1024 entrada.txt
== ESTADO DE LA MEMORIA ==
Pool size: 1024 bytes
OFFSET  SIZE  END    STATE
0       50    49     USED
50      50    99     FREE
100     200   299    USED
300     724   1023   FREE

Variables activas:
C -> [offset=0, size=50]
B -> [offset=100, size=200]

== ESTADO DE LA MEMORIA ==
Pool size: 1024 bytes
OFFSET  SIZE  END    STATE
0       50    49     USED
50      50    99     FREE
100     200   299    USED
300     300   599    USED
600     424   1023   FREE

Variables activas:
D -> [offset=300, size=300]
C -> [offset=0, size=50]
B -> [offset=100, size=200]

[WARN] REALLOC no encontró espacio contiguo; expandiendo pool...
[INFO] Pool expandido a 1144 bytes (+120)

```

Figura 1: Ejecución del simulador con estrategia First-Fit (pool inicial: 1024 bytes)

Análisis del primer estado (fragmentación):

El primer estado del sistema muestra claramente el fenómeno de fragmentación externa. Después de liberar la variable A (originalmente de 100 bytes en el offset 0) y asignar la variable C de 50 bytes, se observa:

- Bloque USADO en offset 0-49 (50 bytes) - Variable C
- Bloque LIBRE en offset 50-99 (50 bytes) - Fragmento residual
- Bloque USADO en offset 100-299 (200 bytes) - Variable B
- Bloque LIBRE en offset 300-1023 (724 bytes) - Espacio disponible

Este estado ilustra perfectamente la fragmentación: existe un bloque libre de 50 bytes que, aunque disponible, está *atrapado* entre bloques ocupados y podría no ser útil para asignaciones futuras de mayor tamaño.

Análisis del segundo estado (fuga de memoria):

Después de asignar las variables D y E, y liberar únicamente E, el sistema muestra:

- Variables C (50 bytes), B (200 bytes) y D (300 bytes) permanecen asignadas
- El fragmento de 50 bytes persiste en offset 50-99
- Espacio libre al final: 424 bytes

La variable D representa una fuga de memoria simulada: fue asignada pero nunca liberada, consumiendo 300 bytes que ya no son accesibles pero tampoco están disponibles para el sistema.

Comportamiento de expansión:

Al intentar realizar REALLOC de B de 200 a 260 bytes, First-Fit encuentra que:

- No hay espacio contiguo suficiente después del bloque de B (el siguiente bloque es D)
- El sistema emite una advertencia y expande el pool
- El pool crece de 1024 a 1144 bytes (+120 bytes)
- La variable B debe ser reubicada en el nuevo espacio

Características observadas de First-Fit:

- Velocidad: Las asignaciones son rápidas al detenerse en el primer bloque adecuado
- Fragmentación: Tiende a acumular pequeños fragmentos al inicio del pool
- Eficiencia espacial: Moderada, con desperdicio en fragmentos pequeños

10.3. Prueba 2: Estrategia Best-Fit

La segunda prueba ejecuta el simulador con la estrategia Best-Fit sobre un pool inicial más grande de 2048 bytes.


```

samircabrera@Samirs-Mac-mini Code % ./memsim best 2048 entrada.txt
== ESTADO DE LA MEMORIA ==
Pool size: 2048 bytes
OFFSET  SIZE  END    STATE
0       50    49     USED
50      50    99     FREE
100     200   299    USED
300     1748  2047   FREE

Variables activas:
C -> [offset=0, size=50]
B -> [offset=100, size=200]

== ESTADO DE LA MEMORIA ==
Pool size: 2048 bytes
OFFSET  SIZE  END    STATE
0       50    49     USED
50      50    99     FREE
100     200   299    USED
300     300   599    USED
600     1448  2047   FREE

Variables activas:
D -> [offset=300, size=300]
C -> [offset=0, size=50]
B -> [offset=100, size=200]

[WARN] REALLOC no encontró espacio contiguo; expandiendo pool...
[INFO] Pool expandido a 2168 bytes (+120)

```

Figura 2: Ejecución del simulador con estrategia Best-Fit (*test_{worst_fit.png}inicial : 2048bytes*)

Análisis del primer estado:

Con un pool inicial mayor, Best-Fit muestra un comportamiento similar en cuanto a fragmentación:

- Fragmento libre de 50 bytes en offset 50-99
- Variables C (50 bytes) y B (200 bytes) ocupan las mismas posiciones que en First-Fit
- Espacio libre final significativamente mayor: 1748 bytes (vs 724 en First-Fit)

La similitud en los primeros bloques es esperada: para estas asignaciones iniciales, Best-Fit coincide con First-Fit porque los primeros bloques disponibles son también los más pequeños adecuados.

Diferencias en comportamiento:

Aunque el patrón de asignación es similar a First-Fit, Best-Fit:

- Busca el bloque más ajustado para cada asignación

- Minimiza el desperdicio en cada operación individual
- Recorre toda la lista de bloques antes de decidir (más lento)

Expansión del pool:

Similar a First-Fit, al intentar REALLOC de B a 260 bytes:

- No encuentra espacio contiguo adecuado
- Expande el pool de 2048 a 2168 bytes (+120 bytes)
- Reubica la variable B en el nuevo espacio

A pesar de tener el doble de memoria inicial (2048 vs 1024), el problema de espacio contiguo persiste debido a que D está inmediatamente después de B, bloqueando la expansión in-situ.

Características observadas de Best-Fit:

- Eficiencia espacial: Mejor aprovechamiento teórico del espacio
- Velocidad: Más lenta que First-Fit por recorrer toda la lista
- Fragmentación: Genera fragmentos más pequeños y numerosos
- Comportamiento similar a First-Fit en este caso particular

10.4. Prueba 3: Estrategia Worst-Fit

La tercera prueba ejecuta el simulador con la estrategia Worst-Fit sobre un pool inicial reducido de 512 bytes.

```

samircabrera@Samirs-Mac-mini Code % ./memsim worst 512 entrada.txt
== ESTADO DE LA MEMORIA ==
Pool size: 512 bytes
OFFSET  SIZE    END    STATE
0       100     99     FREE
100     200    299     USED
300     50     349     USED
350     162    511     FREE

Variables activas:
  C -> [offset=300, size=50]
  B -> [offset=100, size=200]

[WARN] Sin espacio: expandiendo pool en +600 bytes...
[INFO] Pool expandido a 1112 bytes (+600)
== ESTADO DE LA MEMORIA ==
Pool size: 1112 bytes
OFFSET  SIZE    END    STATE
0       100     99     FREE
100     200    299     USED
300     50     349     USED
350     300    649     USED
650     462   1111     FREE

Variables activas:
  D -> [offset=350, size=300]
  C -> [offset=300, size=50]
  B -> [offset=100, size=200]

[WARN] REALLOC no encontró espacio contiguo; expandiendo pool...
[INFO] Pool expandido a 1232 bytes (+120)

```

Figura 3: Ejecución del simulador con estrategia Worst-Fit (pool inicial: 512 bytes)

Comportamiento diferenciado desde el inicio:

Worst-Fit muestra un comportamiento notablemente diferente a las otras estrategias:

- Bloque LIBRE en offset 0-99 (100 bytes) - Fragmento de A liberado
- Bloque USADO en offset 100-299 (200 bytes) - Variable B
- Bloque USADO en offset 300-349 (50 bytes) - Variable C
- Bloque LIBRE en offset 350-511 (162 bytes)

Diferencia clave: Mientras First-Fit y Best-Fit colocaron C al inicio (offset 0), Worst-Fit la colocó cerca del final (offset 300). Esto se debe a que Worst-Fit:

1. Después de liberar A, tiene dos bloques libres: uno de 100 bytes (offset 0) y uno grande al final
2. Al asignar C (50 bytes), selecciona el bloque MÁS GRANDE disponible

3. Coloca C al final y deja un fragmento residual de 162 bytes
4. El bloque de 100 bytes al inicio queda completamente libre

Primera expansión (asignación de D):

Con un pool inicial de solo 512 bytes, Worst-Fit enfrenta falta de espacio antes:

- Intenta asignar D (300 bytes)
- No hay ningún bloque libre de 300 bytes (solo 100 y 162)
- El sistema expande agresivamente: +600 bytes (1.17x el tamaño original)
- Pool crece de 512 a 1112 bytes
- D se asigna en offset 350-649 (inmediatamente después de C)

Segunda expansión (REALLOC de B):

Al igual que las otras estrategias, el REALLOC de B requiere expansión:

- B debe crecer de 200 a 260 bytes
- El siguiente bloque es C (no libre), impidiendo expansión in-situ
- Pool expande de 1112 a 1232 bytes (+120 bytes)

Características observadas de Worst-Fit:

- Distribución diferente: Coloca bloques pequeños en espacios grandes
- Fragmentos residuales más grandes: 100 y 162 bytes (vs 50 en otras estrategias)
- Expansiones más tempranas: Pool pequeño se agota más rápido
- Mayor crecimiento acumulado: $512 \rightarrow 1232$ bytes (2.4x) vs $1024 \rightarrow 1144$ (1.1x) en First-Fit

11. Conclusiones

1. Se implementó exitosamente un simulador completo de gestión dinámica de memoria que replica el comportamiento de las funciones estándar de C.
2. Las tres estrategias de asignación (First-Fit, Best-Fit, Worst-Fit) fueron implementadas y probadas, demostrando sus características distintivas en términos de velocidad y fragmentación.

3. Se logró demostrar problemas reales de gestión de memoria:
 - Fragmentación externa mediante asignaciones y liberaciones intercaladas.
 - Fugas de memoria dejando variables sin liberar intencionalmente.
4. La expansión dinámica del pool usando `realloc` permite que el simulador se adapte a cargas de trabajo variables sin fallar por falta de espacio.
5. La coalescencia automática de bloques libres adyacentes mejora significativamente la eficiencia del sistema al reducir la fragmentación.
6. La arquitectura modular del código facilita el mantenimiento, testing y extensión del simulador con nuevas funcionalidades.
7. El proyecto proporciona una comprensión práctica profunda de cómo los sistemas operativos reales gestionan la memoria dinámica, más allá de la abstracción que ofrecen las funciones estándar.